14 付録 — 逆引き: やりたいことからコードを探す

「一覧をしぼり込みたい」「Excel に書き出したい」から、書き方を 465 個引く。本文に出てくる所と、合わせて覚えるコードつき

14 付録: 逆引き — やりたいことからコードを探す

この逆引きについて

13 付録「出てくるコードとコマンドの一覧」は、「この `useEffect` は何だっけ？」のように、**コードから意味を引く**ための物です。この 14 付録は、その反対です。「**人員の一覧を条件でしぼり込みたい**」「**工数のデータを Excel に書き出したい**」のような、**やりたいことから、書き方を引く**ための物です。

- **全部で 465 個**の「やりたいこと」を、**23 の分類**に分けています。分類は、引き継いだ人が触る順（道具 → Git → HTML・CSS → TypeScript・React → Python・Django → Excel・テスト・運用）です。下の「分類の一覧」から選ぶか、Web 版（`kosu\html`）で `Ctrl` + `F` を押して、やりたいことの言葉（「しぼる」「並べかえる」「送る」など）で探してください。
- **同じやりたいことでも、言語ごとに書き方が違います。** 例えば「並びから条件に合う物だけを集める」は、TypeScript では `items.filter(…)`、Python では `[r for r in rows if …]`、Django では `member.objects.filter(…)` です。それぞれの言語の分類に載せています。
- 「**書き方**」**の列:** そのまま使える形の、短いコードです。`…` の所は、自分の場面に合わせて書きます。このアプリのモデル（`member`・`Business_Time_graph` など）を使った例も多く入れています。
- 「**ひとこと**」**の列:** 気をつけることや、結果の例です。くわしい意味は、13 付録の一覧と、本文の「▶ ここで初めて出てくるコード」の箱にあります。
- 「**出てくる所**」**の列:** その書き方が、00〜12 と 15 で初めて出てくる巻と章です（380 個）。
- 「＋ **合わせて覚える**」: 本文で使った書き方と一緒に覚えておくと、保守ですぐ役に立つ書き方です（85 個）。この付録で足しました。本番のアプリに入れる前に、13 付録の一覧や、それぞれの説明書で、くわしい使い方を確かめ、自分のパソコン（ローカル）で試してください。

> **ヒント:** 直したい所がある時は、「何をしたいか」を短い言葉にしてから、この付録を開いてください。1 つの分類の表を上から下まで一度ながめておくと、あとで探す時に思い出しやすくなります。

分類の一覧

分類	数	分類	数
道具を確かめる・フォルダを移る（PowerShell）	21	Python の基本	29
Python の道具（仮想環境・pip）	14	Python の文字・数・並び・辞書・日時・ファイル	32
Django の道具（manage.py）	14	Django（モデル・ORM）	36
npm と画面の開発用サーバー	16	Django（URL・ビュー・セッション・設定・ログ）	25
Git と GitHub（毎日の作業）	26	Django REST framework（API）	15
Git（間違えた時に戻す）	11	Excel を読む・書く（openpyxl・pandas）	13
HTML（画面の中身）	21	重い処理を裏で動かす（スレッド・タスク）	6
CSS（画面の見た目）	37	テスト（画面: Vitest と Testing Library）	20
TypeScript の基本	31	テスト（サーバー: Django）	11
TypeScript でよく使う道具（並び・文字・数・日付）	31	データベース（psql・SQL）	9
React（部品・状態・画面の切り替え）	29	運用・デプロイ（本番に出す）	6
サーバーとの通信（axios）	12		

道具を確かめる・フォルダを移る（PowerShell）

ターミナル（PowerShell）で、道具の版を確かめたり、フォルダを移ったりします。

やりたいこと	書き方	ひとこと	出てくる所
Git の版を見る	`git --version`		01 3. 道具をそろえる②：Git
Python の版を見る	`python --version`	3.12.x なら OK	01 4. 道具をそろえる③：Python 3.12
Node.js と npm の版を見る	`node --version` `npm --version`	v20 か v22	01 5. 道具をそろえる④：Node.js v22
PostgreSQL の道具の版を見る	`psql --version`		01 6. 道具をそろえる⑤：PostgreSQL
VS Code で今のフォルダを開く	`code .`		01 7. ソースコードを手元に持ってくる
今いるフォルダを知る	`pwd`		00a 4. コマンド — 命令の言葉
フォルダを移る	`cd hozen-kosu-React`	cd .. で 1 つ上へ	00a 4. コマンド — 命令の言葉
フォルダの中を見る	`ls`	正式な名前は Get-ChildItem	00a 4. コマンド — 命令の言葉
隠しファイル（.env など）も見る	`Get-ChildItem -Force`		＋ 合わせて覚える
フォルダを作る	`mkdir "$HOME\Documents\git-projects" -Force`	-Force で、あってもエラーにしない	01 7. ソースコードを手元に持ってくる
ファイルの中を見る	`Get-Content web_console.log`		＋ 合わせて覚える
ファイルの最後の 50 行を見る	`Get-Content web_console.log -Tail 50`	ログの確かめに便利	＋ 合わせて覚える
ログを見張って、増えた行を出し続ける	`Get-Content web_console.log -Wait -Tail 20`	Ctrl＋C で止める	＋ 合わせて覚える
ファイルの中から言葉を探す	`Select-String -Path web_console.log -Pattern "ERROR"`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
ファイルを写す	`Copy-Item .env .env.bak`	書き換える前の控え	＋ 合わせて覚える
ファイルを消す	`Remove-Item 古いファイル.xlsx`	ごみ箱に入らないので注意	＋ 合わせて覚える
空白の入った場所のプログラムを動かす	`& "C:\Program Files\PostgreSQL\16\bin\psql.exe" --version`	先頭の & が要る	01 12. バックエンド準備④：データベースを作ってmigr…
環境変数を入れる（この窓だけ）	`$env:DEBUG = "True"`	窓を閉じると消える	＋ 合わせて覚える
スクリプトの実行を許す（自分だけ）	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`	Activate.ps1 が動かない時	＋ 合わせて覚える
動いているプログラムを止める	`Ctrl ＋ C`	runserver・npm start を止める	＋ 合わせて覚える
前に打ったコマンドを呼び出す	`↑ キー`		＋ 合わせて覚える

Python の道具（仮想環境・pip）

サーバー側の部品は、このアプリ専用の入れ物（仮想環境 `venv`）に入れて使います。

やりたいこと	書き方	ひとこと	出てくる所
仮想環境を作る	`python -m venv venv`	venv フォルダができる	01 8. バックエンド準備①：仮想環境
仮想環境に入る	`.\venv\Scripts\Activate.ps1`	行の頭に (venv) と出る	01 8. バックエンド準備①：仮想環境
仮想環境から出る	`deactivate`		01 8. バックエンド準備①：仮想環境
pip の版と場所を見る	`pip --version`	venv の中か確かめる	01 4. 道具をそろえる③：Python 3.12
部品を全部入れる	`pip install -r requirements.txt`		01 9. バックエンド準備②：requirements.txt…

やりたいこと	書き方	ひとこと	出てくる所
部品を 1 つ足す	`pip install 部品の名前`	足したら requirements.txt にも書く	01 9. バックエンド準備②：requirements.txt…
部品の版を決めて入れる	`pip install Django==4.2.7`	== で版を決める	01 9. バックエンド準備②：requirements.txt…
入っている部品の一覧	`pip list`		01 9. バックエンド準備②：requirements.txt…
古くなった部品を調べる	`pip list --outdated`		＋ 合わせて覚える
入っている部品を書き出す	`pip freeze > requirements.txt`	今の版を全部書く（要らない物も入るので確かめる）	＋ 合わせて覚える
部品を消す	`pip uninstall xlwings`	requirements.txt からも消す	11 12. 不要な依存を安全に整理する手順
部品の説明（版・依存）を見る	`pip show django`		＋ 合わせて覚える
Python の短いプログラムをその場で動かす	`python -c "print(1 + 2)"`		01 10. バックエンド準備③：.env を作る
秘密の鍵（SECRET_KEY）を作る	`python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`	.env に書く	01 10. バックエンド準備③：.env を作る

Django の道具（manage.py）

`manage.py` があるフォルダ（`hozen-kosu-React/`）で、仮想環境に入ってから打ちます。

やりたいこと	書き方	ひとこと	出てくる所
開発用のサーバーを動かす	`python manage.py runserver`	http://localhost:8000	01 14. バックエンドを起動する

やりたいこと	書き方	ひとこと	出てくる所
別の番号（ポート）で動かす	`python manage.py runserver 8001`	8000 が使用中の時	＋ 合わせて覚える
裏方の係（django-q）を動かす	`python manage.py qcluster`		01 9. バックエンド準備②：requirements.txt…
モデルの変更から設計図を作る	`python manage.py makemigrations`		08 21. 保守の心得
設計図をデータベースに当てる	`python manage.py migrate`		01 12. バックエンド準備④：データベースを作って migr…
当てた・まだの設計図を見る	`python manage.py showmigrations`		＋ 合わせて覚える
設計図の SQL を見る	`python manage.py sqlmigrate kosu 0001`	本番の前の確かめ	＋ 合わせて覚える
管理画面の利用者を作る	`python manage.py createsuperuser`		01 13. バックエンド準備⑤：管理者を作る
静的ファイルを集める（本番）	`python manage.py collectstatic`	staticfiles/ に集まる	09 3. 日常運用コマンド集
テストを全部流す	`python manage.py test --settings=hozen_another.test_settings`		12 13. テストの実行コマンドと出力例
テストを 1 つのクラスだけ流す	`python manage.py test kosu.tests.test_backend.TestModels --settings=hozen_another.test_settings`		12 13. テストの実行コマンドと出力例
Django の中で Python を打つ	`python manage.py shell`	モデルを直接さわって確かめる	15 5. 不具合
設定の誤りを調べる	`python manage.py check`		11 12. 不要な依存を安全に整理する手順
本番向けの設定の誤りを調べる	`python manage.py check --deploy`		＋ 合わせて覚える

npm と画面の開発用サーバー

画面側（React）は `frontend/` フォルダで打ちます。

やりたいこと	書き方	ひとこと	出てくる所
画面のフォルダへ移る	`cd frontend`		01 16. フロントエンド準備①：package.json 全…
部品を全部入れる	`npm install`	package.json のとおり	01 16. フロントエンド準備①：package.json 全…
package-lock.json のとおりに入れる	`npm ci`	版がそろう（本番・CI 向け）	15 6. 開発・運用の整理
部品を足す	`npm install dayjs`	dependencies に入る	01 16. フロントエンド準備①：package.json 全…
開発の時だけの部品を足す	`npm install -D vitest`	devDependencies に入る	＋ 合わせて覚える
部品を消す	`npm uninstall @fullcalendar/daygrid`		11 12. 不要な依存を安全に整理する手順
開発用サーバーを動かす	`npm start`	http://localhost:3000	01 18. フロントエンドを起動する
本番用に作る	`npm run build`	frontend/build/ にできる	01 19. 本番ビルド
テスト（見張り続ける）	`npm test`	q で終わる	09 4. テストの実行
テストを 1 回だけ流す	`npm run test:run`	直した後・提出前に	09 4. テストの実行
テストの結果を画面で見る	`npm run test:ui`		09 4. テストの実行
古くなった部品を調べる	`npm outdated`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
弱点のある部品を調べる	`npm audit`		＋ 合わせて覚える
部品が壊れた時に入れ直す	`Remove-Item -Recurse -Force node_modules` `npm install`	package-lock.json は残す	＋ 合わせて覚える
登録したコマンドの一覧	`"scripts": { "start": "react-scripts start", … }`	package.json	01 16. フロントエンド準備①：package.json 全…
動くのに要る部品の一覧	`"dependencies": { "axios": "^1.11.0", … }`	^ は「上の版でもよい」	01 16. フロントエンド準備①：package.json 全…

Git と GitHub（毎日の作業）

直す前に `git pull`、直したら `add` → `commit` → `push` が、毎日の流れです。

やりたいこと	書き方	ひとこと	出てくる所
最初に名前とメールを登録	`git config --global user.name "Yamada Taro"` `git config --global user.email "you@example.com"`		01 3. 道具をそろえる②：Git
リポジトリを写してくる	`git clone https://github.com/your-company/hozen-kosu-React.git`		01 7. ソースコードを手元に持ってくる
今どうなっているか	`git status`	まず打つ	02 3. このプロジェクトの Git の状態を見てみる
短く表示	`git status -s`		02 4. git status — 今どうなっているか
変えた所を見る	`git diff`		02 5. git diff — 何をどう変えたか

やりたいこと	書き方	ひとこと	出てくる所
add 済みの変更を見る	`git diff --staged`		02 5. git diff — 何をどう変えたか
次のコミットに入れる	`git add frontend/src/MainPage/Help.tsx`		02 6. git add — 記録の準備
全部の変更を入れる	`git add .`	.env などが入らないか先に status で確かめる	02 6. git add — 記録の準備
記録する（コミット）	`git commit -m "人員一覧で異動・退社を後ろに変更"`	何をしたかを書く	02 7. git commit — 記録を確定する
GitHub へ送る	`git push origin master`		02 9. リモートとの同期：push / pull / fet…
GitHub の新しい物を取り込む	`git pull origin master`	作業の前に	02 9. リモートとの同期：push / pull / fet…
取ってくるだけ（混ぜない）	`git fetch origin`		02 9. リモートとの同期：push / pull / fet…
歴史を見る	`git log --oneline`		02 8. git log — 過去の記録を読む
最新の 5 件	`git log --oneline -5`		02 8. git log — 過去の記録を読む
枝分かれを絵で見る	`git log --oneline --graph --all`		02 8. git log — 過去の記録を読む
1 つのコミットの中身を見る	`git show 9156f15`		02 8. git log — 過去の記録を読む
ファイルごとの変更の歴史	`git log -p -- kosu/models.py`		＋ 合わせて覚える
その行を最後に変えたのはだれか	`git blame kosu/models.py`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
つながっている GitHub を見る	`git remote -v`		02 3. このプロジェクトの Git の状態を見てみる
ブランチの一覧	`git branch -a`		02 3. このプロジェクトの Git の状態を見てみる
ブランチを作って移る	`git switch -c help-fix`		02 10. ブランチ — 作業の枝分かれ
ブランチを移る	`git switch master`		02 10. ブランチ — 作業の枝分かれ
ブランチを取り込む	`git merge help-fix`	master にいる時に	02 11. merge
ブランチを消す	`git branch -d help-fix`		02 11. merge
Git に入れないファイルを決める	`.gitignore に .env と書く`	秘密・大きな物は入れない	＋ 合わせて覚える
GitHub のパスワードを覚えさせる	`git config --global credential.helper manager`		02 15. トークン認証 — GitHub への安全なログイン

Git（間違えた時に戻す）

戻す前に、必ず `git status` で今の様子を見ます。`--hard` は消えた物が戻らないことがあります。

やりたいこと	書き方	ひとこと	出てくる所
add を取り消す	`git restore --staged Help.tsx`	ファイルの中身はそのまま	02 6. git add — 記録の準備
ファイルの変更を捨てる	`git restore frontend/src/MainPage/Help.tsx`	元に戻せない	02 6. git add — 記録の準備
直前のコミットのメッセージを直す	`git commit --amend -m "正しいメッセージ"`	push 前だけ	02 16. 復旧コマンドと「やってしまった」対処集

やりたいこと	書き方	ひとこと	出てくる所
コミットを取り消す（変更は add 済みで残す）	`git reset --soft HEAD~1`		02 16. 復旧コマンドと「やってしまった」対処集
コミットと add を取り消す（変更は残す）	`git reset --mixed HEAD~1`		02 16. 復旧コマンドと「やってしまった」対処集
全部消して戻す	`git reset --hard HEAD~1`	変更が消える	02 16. 復旧コマンドと「やってしまった」対処集
push 済みのコミットを打ち消す	`git revert 9156f15`	打ち消すコミットを新しく作る（歴史は消さない）	＋ 合わせて覚える
消えたコミットを探す	`git reflog`		02 16. 復旧コマンドと「やってしまった」対処集
誤って入れた .env を Git から外す	`git rm --cached .env`	ファイルは残る	02 14. .gitignore ― 追跡しないファイルの指定
作業を一時しまう	`git stash` `git stash pop`	pull する前に	＋ 合わせて覚える
ぶつかり（コンフリクト）を直した後	`git add ファイル` `git commit`	<<<<<<< の印を消してから	＋ 合わせて覚える

HTML（画面の中身）

React の画面（JSX）も、元は HTML のタグです。

やりたいこと	書き方	ひとこと	出てくる所
見出し	`<h1>工数入力</h1>`	h1〜h6	03 0. 前提知識：ブラウザが画面を作るしくみ
段落	`<p>…</p>`		03 0. 前提知識：ブラウザが画面を作るしくみ

やりたいこと	書き方	ひとこと	出てくる所
リンク	`<a href="/help">ヘルプ</a>`		03 1. HTMLの基本
新しいタブで開くリンク	`<a href="/help" target="_blank">ヘルプ</a>`		03 1. HTMLの基本
画像	`<img src="logo.png" alt="ロゴ">`	alt は画像の説明	00 5. 実コード①：Login.tsx
囲み（意味なし）	`<div class="container">…</div>`		00 5. 実コード①：Login.tsx
文の一部を囲む	`<span>…</span>`		03 1. HTMLの基本
箇条書き	`<ul><li>…</li></ul>`	番号付きは	03 1. HTMLの基本
表	`<table><thead><tr><th>日付</th></tr></thead><tbody>…</tbody></table>`		03 1. HTMLの基本
表のマスを横につなぐ	`<td colspan="4">`		＋ 合わせて覚える
入力欄	`<input type="text" id="numberInput">`		00 5. 実コード①：Login.tsx
入力欄の名前	`<label for="numberInput">従業員番号</label>`	for と id をそろえる	00 5. 実コード①：Login.tsx
日付の入力欄	`<input type="date">`		03 11. 実コード全解説：global.css 全342行を…
選ぶ欄（プルダウン）	`<select><option value="1">1直</option></select>`	value が送られる値	03 1. HTMLの基本
何行も書ける入力欄	`<textarea rows="5"></textarea>`		03 1. HTMLの基本
ボタン	`<button type="button">検索</button>`		00 5. 実コード①：Login.tsx
押せないボタン	`<button disabled>…</button>`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
読み上げソフトに「警告」と伝える	`<div role="alert">…</div>`		00 5. 実コード①：Login.tsx
横線	`<hr>`		03 1. HTMLの基本
改行	` `		03 1. HTMLの基本
コメント	`<!-- … -->`	画面には出ない	03 1. HTMLの基本

CSS（画面の見た目）

このアプリは、画面ごとの `○○.module.css`（CSS Modules）と、共通の `global.css` を使います。

やりたいこと	書き方	ひとこと	出てくる所
クラスに見た目を付ける	`.blue_button { … }`		03 7. Flexbox
中にある物を選ぶ	`.search-bar input { … }`		03 11. 実コード全解説：global.css 全342行を…
属性で選ぶ	`input[type="date"] { … }`		03 11. 実コード全解説：global.css 全342行を…
マウスが乗った時	`.blue_button:hover { … }`		03 8. 擬似クラスと擬似要素
入力中の欄	`input:focus { … }`		03 8. 擬似クラスと擬似要素
押せないボタン	`button:disabled { … }`		03 8. 擬似クラスと擬似要素
1 行おきに色を変える	`tr:nth-child(even) { … }`		03 8. 擬似クラスと擬似要素
前に飾りを足す	`.icon::before { content: "⚠️ "; }`		03 8. 擬似クラスと擬似要素
文字の色・背景の色	`color: #333;` `background-color: #007bff;`		03 4. CSSの基本文法

やりたいこと	書き方	ひとこと	出てくる所
文字の大きさ・太さ	`font-size: 16px;` `font-weight: bold;`		03 4. CSSの基本文法
文字を真ん中にそろえる	`text-align: center;`		03 11. 実コード全解説：global.css 全342行を…
内側・外側の余白	`padding: 10px;` `margin: 0 auto;`	auto で横の真ん中	03 11. 実コード全解説：global.css 全342行を…
枠線と角の丸み	`border: 1px solid #ccc;` `border-radius: 8px;`		03 11. 実コード全解説：global.css 全342行を…
幅と高さ	`width: 100%;` `height: 100%;`		03 10. メディアクエリ
幅の上限・下限	`max-width: 1200px;` `min-width: unset;`		03 10. メディアクエリ
横に並べる	`display: flex;` `gap: 10px;`		03 7. Flexbox
横の真ん中・縦の真ん中	`justify-content: center;` `align-items: center;`		03 7. Flexbox
隠す	`display: none;`		03 7. Flexbox
画面の決まった所に置く（スクロールしても動かない）	`position: fixed;` `bottom: 30px;` `right: 30px;`		03 11. 実コード全解説：global.css 全342行を…
親を基準に好きな所へ置く	`position: absolute;` `top: 0;` `left: 0;`	親は position: relative	03 11. 実コード全解説：global.css 全342行を…
重なりの上下	`z-index: 1;`	大きいほど手前	03 11. 実コード全解説：global.css 全342行を…
はみ出したらスクロール	`overflow-x: auto;`		03 11. 実コード全解説：global.css 全342行を…

やりたいこと	書き方	ひとこと	出てくる所
影	`box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);`		03 11. 実コード全解説：global.css 全342行を…
半透明の色	`rgba(0, 0, 0, 0.1)`		03 6. 単位と色
なめらかに変える	`transition: background-color 0.3s ease;`		03 9. トランジションとアニメーション
動きを付ける	`@keyframes fadeInUp { from { … } to { … } }` `animation: fadeInUp 1s ease-out;`		03 9. トランジションとアニメーション
回す・ずらす	`transform: rotate(45deg);` `transform: translateY(50px);`		03 9. トランジションとアニメーション
画面の幅で大きさを変える	`clamp(1.5rem, 1.5rem + 3vw, 4rem)`		03 11. 実コード全解説：global.css 全342行を…
計算した幅	`width: calc(100% - 40px);`		03 12. 実コード全解説：Login.module.css …
画面の高さいっぱい	`min-height: 100vh;`		03 11. 実コード全解説：global.css 全342行を…
共通の色を変数にする	`:root { --main-color: #007bff; }` `color: var(--main-color);`		03 11. 実コード全解説：global.css 全342行を…
スマホの幅の時だけ	`@media (max-width: 768px) { … }`		03 10. メディアクエリ
表の線を 1 本にまとめる	`border-collapse: collapse;`		03 11. 実コード全解説：global.css 全342行を…
長い言葉を折り返す	`word-break: break-all;`		03 11. 実コード全解説：global.css 全342行を…

やりたいこと	書き方	ひとこと	出てくる所
画像を枠に収める	`object-fit: contain;`		03 12. 実コード全解説：Login.module.css …
いちばん強くする	`display: none !important;`	使いすぎない	03 13. 実コード追加解説：KosuList / MainM…
日付欄のカレンダーの印を消す	`input[type=date]::-webkit-calendar-picker-indicator { … }`		03 11. 実コード全解説：global.css 全342行を…

TypeScript の基本

型（値の種類）を書いておくと、まちがいを、動かす前にエディターが教えてくれます。

やりたいこと	書き方	ひとこと	出てくる所
変えない値に名前を付ける	`const pageSize = 20;`		00 5. 実コード①：Login.tsx
あとで変える値	`let count = 0;`		04 2. 変数 — 値に名前を付ける
型を書く	`let age: number = 42;`		04 14. TypeScript — 型という安全装置
文字・数・真偽	`string` `number` `boolean`		00 5. 実コード①：Login.tsx
どちらかの型	`value: string | number`		04 14. TypeScript — 型という安全装置

やりたいこと	書き方	ひとこと	出てくる所
無いかもしれない	`string | null`		04 14. TypeScript ― 型という安全装置
オブジェクトの形に名前を付ける	`interface Employee { employeeNo: number; name: string; }`		04 14. TypeScript ― 型という安全装置
無くてもよい項目	`buttonColor?: string;`		05 3. props ― 親から子へ渡す設定値
型に名前を付ける	`type Props = { isLoading: boolean };`		04 14. TypeScript ― 型という安全装置
キーが何でもよい辞書	`{ [key: string]: string }`		00 10. 実コード⑥：index.tsx
並び（配列）の型	`string[]`		05 4. state と useState ― 画面が覚える値
関数を作る	`const getDayOfWeek = (dateStr: string): string => { … };`		04 19. 実コード全解説：KosuList.tsx の型と関…
条件で分ける	`if (…) { … } else if (…) { … } else { … }`		00 5. 実コード①：Login.tsx
値ごとに分ける	`switch (value) { case "1": … break; default: … }`		04 6. 条件分岐 ― if / else / 三項演算子 /…
条件で値を選ぶ	`a ? b : c`		00 5. 実コード①：Login.tsx
同じか（型も）	`a === b` `a !== b`	== は使わない	00 5. 実コード①：Login.tsx

やりたいこと	書き方	ひとこと	出てくる所

かつ・または・ではない	`a && b`
	`a || b`
	`!a`
			00 5. 実コード①：Login.tsx

無ければ右を使う	`value ?? ""`	null・undefined の時だけ	00 5. 実コード①：Login.tsx
無ければ止まらずに undefined	`err.response?.data`		00 5. 実コード①：Login.tsx
決まった回数くり返す	`for (let i = 0; i < 5; i++) { … }`		04 7. 繰り返し — for とループ
並びを 1 つずつ	`for (const item of items) { … }`		04 7. 繰り返し — for とループ
文字の中に値を入れる	`` `${year}-${month}` ``		04 5. テンプレートリテラル — 文字列に値を埋め込む
中身を広げる	`{ ...base, filter: true }` `[...arr1, 3, 4]`		04 13. 分割代入・スプレッド・オプショナルチェイニング
取り出して名前を付ける	`const { data } = response;` `const [a, b] = arr;`		00 5. 実コード①：Login.tsx
エラーを受け止める	`try { … } catch (err) { … } finally { … }`		00 5. 実コード①：Login.tsx
完了を待つ関数	`const fetchData = async () => { const res = await api.get(…); }`		00 5. 実コード①：Login.tsx
外から使えるようにする	`export default Login;`		00 5. 実コード①：Login.tsx
ほかのファイルから読む	`import api from "../api/axios";`		00 5. 実コード①：Login.tsx
何の型でもよい（使いすぎない）	`any`	テストのにせ物など	12 4. 実コード全解説：Login.test.tsx 全13…

やりたいこと	書き方	ひとこと	出てくる所
型をこれだと伝える	`e.target as HTMLInputElement`		00 5. 実コード①：Login.tsx
コメント	`// 1 行` `/* 何行も */`	JSX の中は {/ … /}	00 5. 実コード①：Login.tsx

TypeScript でよく使う道具（並び・文字・数・日付）

一覧の画面で、データを並べかえたり、しぼったり、形を整えたりする時に使います。

やりたいこと	書き方	ひとこと	出てくる所
並びの数	`items.length`		04 10. 配列 — 値を並べて持つ
1 つずつ形を変えた並び	`items.map((item) => …)`	JSX の一覧に使う	04 11. 配列の必殺技 — map と filter
条件に合う物だけ	`items.filter((item) => item.shop === "P")`		04 11. 配列の必殺技 — map と filter
最初に合う 1 つ	`items.find((item) => item.id === id)`		＋ 合わせて覚える
入っているか	`items.includes("P")`		＋ 合わせて覚える
1 つでも合うか・全部合うか	`items.some(…)` `items.every(…)`		＋ 合わせて覚える
合計を出す	`items.reduce((sum, x) => sum + x.time, 0)`		＋ 合わせて覚える
並べかえる	`items.sort((a, b) => a.id - b.id)`	元の並びも変わる	＋ 合わせて覚える
後ろに足す	`items.push(x)`		04 10. 配列 — 値を並べて持つ
文字で区切って並びに	`"08:30".split(":")`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
並びを文字でつなぐ	`list.join(" / ")`		＋ 合わせて覚える
前後の空白を取る	`text.trim()`		＋ 合わせて覚える
前を 0 でうめる	`String(n).padStart(2, "0")`	5 → "05"	＋ 合わせて覚える
一部を取り出す	`text.slice(0, 4)`		＋ 合わせて覚える
置きかえる	`text.replace("-", "/")`	全部なら replaceAll	＋ 合わせて覚える
文字を数に	`Number(employee_no)`	数でない時は NaN	＋ 合わせて覚える
小数を切り上げる	`Math.ceil(count / pageSize)`	ページの数	04 19. 実コード全解説：KosuList.tsx の型と関…
小数を切り捨てる・四捨五入	`Math.floor(x)` `Math.round(x)`		＋ 合わせて覚える
数か調べる	`Number.isNaN(n)`		＋ 合わせて覚える
日付を作る	`new Date(dateStr)`		04 19. 実コード全解説：KosuList.tsx の型と関…
曜日（0＝日曜）	`date.getDay()`		04 19. 実コード全解説：KosuList.tsx の型と関…

年・月・日	`date.getFullYear()`
	`date.getMonth() + 1`
	`date.getDate()`
		月は 0 から	＋ 合わせて覚える

日付を YYYY-MM-DD に	`date.toISOString().slice(0, 10)`	時差でずれることがある	＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
軽い日付の道具で形を整える	`dayjs(date).format("YYYY-MM-DD")`	dayjs を使う	＋ 合わせて覚える
オブジェクトを JSON の文字に	`JSON.stringify(obj)`		＋ 合わせて覚える
JSON の文字をオブジェクトに	`JSON.parse(text)`		＋ 合わせて覚える
キーの一覧	`Object.keys(obj)`		＋ 合わせて覚える
少したってから動かす	`setTimeout(() => { … }, 3000)`	ミリ秒	05 10. 実コード全解説：Loading.tsx 全42行
くり返し動かす（見張り）	`const id = setInterval(check, 2000);` `clearInterval(id);`	タスクの状態を見に行く時など	＋ 合わせて覚える
画面に出す（調べ物）	`console.log(value)`	ブラウザーの F12 → Console	＋ 合わせて覚える
タブの名前を変える	`document.title = "ログイン － 業務工数システム";`		00 10. 実コード⑥：index.tsx

React（部品・状態・画面の切り替え）

画面は「部品（コンポーネント）」の組み合わせです。値が変わると、React が画面を描き直します。

やりたいこと	書き方	ひとこと	出てくる所
部品を作る	`const Login: React.FC = () => { return (<div>…</div>); };`		00 5. 実コード①：Login.tsx
覚えておく値（状態）	`const [employee_no, setNumber] = useState<string>("");`		00 5. 実コード①：Login.tsx
値を変えて描き直す	`setNumber(e.target.value)`		00 5. 実コード①：Login.tsx
表示の後に動かす	`useEffect(() => { … }, [])`	[] で最初の 1 回だけ	00 10. 実コード⑥：index.tsx

やりたいこと	書き方	ひとこと	出てくる所
値が変わるたびに動かす	`useEffect(() => { … }, [location])`		00 10. 実コード⑥：index.tsx
片付け（部品が消える時）	`useEffect(() => { … return () => clearInterval(id); }, [])`		00 10. 実コード⑥：index.tsx
描き直さずに覚える・部品を指す	`const searchBarRef = useRef<HTMLDivElement>(null);`		05 6. useRef — 再描画を起こさない箱
ref の中身	`searchDayRef.current`		05 5. useEffect — タイミングで処理を走らせる
入力欄と値を結ぶ	`<input value={employee_no} onChange={(e) => setNumber(e.target.value)} />`		00 5. 実コード①：Login.tsx
最初の値だけ入れる入力欄	`<input defaultValue={searchDayRef.current} />`		05 8. 実コード全解説：KosuList.tsx 全215行
押した時	`<button onClick={() => handleSearch(true)}>検索</button>`		04 9. アロー関数 — 関数の短い書き方
キーを押した時	`onKeyDown={(e) => { if (e.key === "Enter") … }}`		00 5. 実コード①：Login.tsx
フォームを送った時	`<form onSubmit={handleSubmit}>` `e.preventDefault();`	画面の読み直しを止める	00 5. 実コード①：Login.tsx
条件が合う時だけ出す	`{errorMessage && (<div role="alert">{errorMessage}</div>)}`		00 5. 実コード①：Login.tsx
どちらかを出す	`{loading ? <Loading isLoading={loading} /> : <TableContainer …>}`		＋ 合わせて覚える
読み込み中は先に返す	`if (loading) return <div><Loading isLoading={loading} /></div>;`		05 8. 実コード全解説：KosuList.tsx 全215行
一覧を並べる	`{items.map((item) => <tr key={item.id}>…</tr>)}`	key が要る	04 19. 実コード全解説：KosuList.tsx の型と関…

やりたいこと	書き方	ひとこと	出てくる所
クラスを付ける（CSS Modules）	`className={styles["login-wrapper"]}`		00 5. 実コード①：Login.tsx
直接スタイルを書く	`style={{ marginTop: 8 }}`		05 9. 実コード全解説：Pagination.tsx 全71行
押せなくする	`disabled={currentPage === 1}`		05 9. 実コード全解説：Pagination.tsx 全71行
部品に値を渡す	`<Loading isLoading={loading} />`		05 8. 実コード全解説：KosuList.tsx 全215行
画像を取り込んで出す	`import logo from "../img/TitleRogo.png";` `<img src={logo} alt="…" />`		00 5. 実コード①：Login.tsx
画面と URL を結ぶ	`<Route path="/kosu-new" element={<KosuNew />} />`		00 10. 実コード⑥：index.tsx
URL の中の番号を受け取る	`const { id } = useParams();`	path="/kosu-update/:id" の :id	＋ 合わせて覚える
プログラムで画面を移る	`const navigate = useNavigate();` `navigate("/");`		00 5. 実コード①：Login.tsx
リンクで画面を移る	`<Link to="/help">ヘルプ</Link>`		00 5. 実コード①：Login.tsx
今の URL を知る	`const location = useLocation();` `location.pathname`		00 10. 実コード⑥：index.tsx
アプリ全体を画面に描く	`ReactDOM.createRoot(document.getElementById("root") as HTMLElement).render(<App />)`		00 10. 実コード⑥：index.tsx
中身（子）を受け取る部品	`const Box = ({ children }: { children: React.ReactNode }) => <div>{children}</div>;`		05 11. 実コード全解説：TableContainer.ts…

サーバーとの通信（axios）

画面は `api`（`axios.create` で作った係）で、Django の API とやりとりします。

やりたいこと	書き方	ひとこと	出てくる所
通信の係を作る	`const api = axios.create({ baseURL: process.env.REACT_APP_API_BASE_URL, withCredentials: true });`		04 17. 実コード全解説：axios.ts 全8行
環境変数を読む	`process.env.REACT_APP_API_BASE_URL`	REACT_APP_ で始まる名前だけ読める	04 17. 実コード全解説：axios.ts 全8行
データを取ってくる	`const response = await api.get("/api/member_list/", { params: { page: 1 } });`	params は ?page=1 になる	04 19. 実コード全解説：KosuList.tsx の型と関…
データを送る	`await api.post("/api/login/", { employee_no: Number(employee_no) });`		00 5. 実コード①：Login.tsx
データを直す・消す	`await api.put(`/api/member_update/${employeeNo}/`, formData)` `await api.delete(…)`		＋ 合わせて覚える
返事の中身	`response.data`		00 5. 実コード①：Login.tsx
返事の番号	`response.status`	200 成功・400 入力の誤り・401 ログインしていない	＋ 合わせて覚える
通信のエラーか調べる	`if (axios.isAxiosError(err)) { … }`		00 5. 実コード①：Login.tsx
エラーの返事の中身	`err.response?.data.message`		00 5. 実コード①：Login.tsx
ログインしていない時にログイン画面へ	`if (err.response?.status === 401) navigate("/login");`		00 5. 実コード①：Login.tsx
ファイルを送る	`const form = new FormData();` `form.append("file", file);` `await api.post(url, form);`		＋ 合わせて覚える
ファイルを受け取って保存	`api.get(url, { responseType: "blob" })`	Excel のダウンロードなど	＋ 合わせて覚える

Python の基本

字下げ（このアプリは空白 2 つ）で、まとまりを表します。字下げがずれると動きません。

やりたいこと	書き方	ひとこと	出てくる所
変数に入れる	`input_number = data.get('employee_no')`	型は書かなくてよい	00 7. 実コード③：main_views.py の Login
関数を作る	`def time_index(time):`		08 8. kosu_utils.py 逐行解説①: time_...
値を返す	`return start_time_hour, start_time_min`	2 つ返すとタプル	＋ 合わせて覚える
いくつでも受け取る	`def save(self, *args, **kwargs):`		00 8. 実コード④：models.py
辞書をばらして渡す	`member.objects.create(…, **break_times)`		12 8. 実コード全解説：test_backend.py の土台
名前を付けて渡す	`models.CharField('氏名', max_length=100)`		00 6. 実コード②：kosu/urls.py
条件で分ける	`if …: … elif …: … else: …`		06 4. 条件分岐 if
条件で値を選ぶ	`a if 条件 else b`		06 8. 三項演算子・f文字列

かつ・または・ではない	`a and b`
	`a or b`
	`not a`
			06 4. 条件分岐 if

入っているか	`'temp_file_path' in locals()` `val not in [None, '']`		08 16. モデル④: kosu_division
無いか	`if obj is None:`		06 4. 条件分岐 if

やりたいこと	書き方	ひとこと	出てくる所
くり返す	`for d in range(1, 8):`		06 5. 繰り返し for
番号つきでくり返す	`for i, e in enumerate(detail_list):`		08 13. kosu_utils.py 逐行解説⑥: deta…
条件の間くり返す	`while 条件:`		＋ 合わせて覚える
くり返しを抜ける・飛ばす	`break` `continue`		06 5. 繰り返し for
並びを作り直す（内包表記）	`[today - datetime.timedelta(days=d) for d in range(1, 8)]`		06 5. 繰り返し for
エラーを受け止める	`try: … except json.JSONDecodeError: …`		00 7. 実コード③：main_views.py の Login
何でも受け止める	`except Exception as e:`	最後の守り	00 7. 実コード③：main_views.py の Login
自分でエラーを起こす	`raise ValueError("未入力箇所があります")`		08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
必ず最後に動かす	`finally: os.remove(temp_file_path)`		15 5. 不具合
クラスを作る	`class member(models.Model):`		00 7. 実コード③：main_views.py の Login
作る時に動く関数	`def __init__(self, name, age):`		06 9. クラスとインスタンス
表示の時の文字	`def __str__(self): return self.name`		00 8. 実コード④：models.py

やりたいこと	書き方	ひとこと	出てくる所
親の処理も動かす	`super().save(*args, **kwargs)`		00 8. 実コード④：models.py
自分と同じクラス	`self.__class__.objects.count()`		00 8. 実コード④：models.py
部品を読み込む	`from django.db import models` `import json`		00 6. 実コード②：kosu/urls.py
開いて自動で閉じる	`with open(path, encoding="utf-8") as f:`		06 7. 関数 def
何もしない	`pass`		06 9. クラスとインスタンス
画面に出す（調べ物）	`print(value)`	本番ではログを使う	06 1. インデント ― Python最大の特徴

Python の文字・数・並び・辞書・日時・ファイル

よく使う型の道具です。

やりたいこと	書き方	ひとこと	出てくる所
文字の中に値を入れる	`f'{member_name}氏の工数未入力があります。'`		06 5. 繰り返し for
前を 0 でうめる	`f"{n:02d}"`	5 → 05	12 9. 実コード全解説：TestModels
区切りで分ける	`post_time.split(':')`		09 7. tasks.py 全解説②：予測データ出力
文字の場所	`post_time.index(':')`	無いとエラー	08 8. kosu_utils.py 逐行解説①: time_...
一部を取り出す	`text[1:5]` `text[-1]`		08 7.【最重要】休憩文字列 #HHMMHHMM の読み方

やりたいこと	書き方	ひとこと	出てくる所
置きかえる	`text.replace('#', '')`		08 7. 【最重要】休憩 文字列 #HHMMHHMM の読み方
前後の空白を取る	`text.strip()`		06 1. インデント ― Python最大の特徴
文字を数に・数を文字に	`int("12")` `str(12)`		06 2. 変数とデータ型
長さ	`len(detail_list)`		07 11. 実コード全解説：asynchronous_view...
後ろに足す	`items.append(x)`		06 3. リスト・辞書・タプル
辞書から読む（無ければ None）	`data.get('employee_no')`		00 7. 実コード③：main_views.py の Login
辞書の中身を全部	`for key, value in d.items():`		15 4. 安全の問題
辞書をまとめて足す	`d.update(other)`		07 5. 実コード全解説：member_views.py 全2...
JSON の文字を辞書に	`json.loads(request.body)`		00 7. 実コード③：main_views.py の Login
辞書を JSON の文字に	`json.dumps(data, ensure_ascii=False)`	日本語をそのまま	12 8. 実コード全解説：test_backend.py の土台
今日の日付	`datetime.date.today()`		06 12. import と相対 import
今の日時（時間帯つき）	`timezone.now()`	Django の道具	＋ 合わせて覚える
何日前・何日後	`today - datetime.timedelta(days=1)`		06 5. 繰り返し for

やりたいこと	書き方	ひとこと	出てくる所
文字を日付に	`datetime.datetime.strptime(start_day, '%Y-%m-%d').date()`		08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
日付を文字に	`day.strftime('%Y-%m-%d')`		06 16. 実コード全解説③：asynchronous_vie…
日本の時間に直す	`value.astimezone(pytz.timezone('Asia/Tokyo'))`		09 9. tasks.py 全解説④：人員・班員・定義・その他…
ファイルの場所をつなぐ	`os.path.join(settings.MEDIA_ROOT, 'backup.xlsx')`		00 12. 実コード⑦：settings.py の主要設定を1…
ファイルがあるか	`os.path.exists(path)`		07 11. 実コード全解説：asynchronous_view…
フォルダを作る	`os.makedirs(media_dir)`	途中も作る	09 6. tasks.py 全解説①：工数バックアップ
ファイルを消す	`os.remove(temp_file_path)`		07 11. 実コード全解説：asynchronous_view…
場所の最後の名前	`os.path.basename(file_path)`		07 11. 実コード全解説：asynchronous_view…
プロジェクトのいちばん上	`Path(__file__).resolve().parent.parent`		00 12. 実コード⑦：settings.py の主要設定を1…
一時ファイルを作る	`tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False)`		06 11. with 文
ファイルを全部の行で読む	`log_file.readlines()`		06 7. 関数 def
環境変数を読む	`os.getenv('DB_NAME', '')`		00 12. 実コード⑦：settings.py の主要設定を1…

やりたいこと	書き方	ひとこと	出てくる所
ほかと重ならない ID	`str(uuid.uuid4())`		06 16. 実コード全解説③：asynchronous_vie…
少し待つ	`time.sleep(3)`		07 11. 実コード全解説：asynchronous_view…

Django（モデル・ORM）

データベースの表を Python のクラス（モデル）で書き、ORM（SQL を書かずに読み書きする仕組み）で使います。

やりたいこと	書き方	ひとこと	出てくる所
表を作る（モデル）	`class member(models.Model):`	変えたら makemigrations → migrate	00 8. 実コード④：models.py
整数の列	`employee_no = models.IntegerField('従業員番号')`		00 8. 実コード④：models.py
文字の列	`name = models.CharField('氏名', max_length=100)`		00 8. 実コード④：models.py
選び肢つきの列	`models.CharField('ショップ', choices=shop_list, max_length=15)`		00 8. 実コード④：models.py
真偽の列	`models.BooleanField('権限')`		00 8. 実コード④：models.py
日付・日時の列	`models.DateField('就業日')` `models.DateTimeField(auto_now_add=True)`		00 8. 実コード④：models.py
ほかの表を指す	`models.ForeignKey(member, verbose_name='氏名', null=True, on_delete=models.SET_NULL)`		00 8. 実コード④：models.py
空でもよい	`null=True, blank=True`	null はデータベース、blank は入力の確かめ	＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
全部	`kosu_division.objects.all()`		00 7. 実コード③：main_views.py の Login
しぼる	`member.objects.filter(employee_no=input_number)`		00 7. 実コード③：main_views.py の Login
除く	`member.objects.exclude(shop='異動・退社')`		15 5. 不具合
1 つだけ取る	`member.objects.get(employee_no=no)`	無いと DoesNotExist	00 7. 実コード③：main_views.py の Login
無かった時を受け止める	`except member.DoesNotExist:`		00 7. 実コード③：main_views.py の Login
1 つ取る（無ければ None）	`member.objects.filter(…).first()`		06 8. 三項演算子・f文字列
最後の 1 つ	`kosu_division.objects.order_by("id").last()`		00 7. 実コード③：main_views.py の Login
あるか	`member.objects.filter(employee_no=no).exists()`		00 7. 実コード③：main_views.py の Login
数える	`AsyncTask.objects.count()`		00 8. 実コード④：models.py
並べかえる	`order_by('-id')`	- で逆	00 7. 実コード③：main_views.py の Login
期間でしぼる	`filter(work_day2__gte=start_day, work_day2__lte=end_day)`		07 5. 実コード全解説：member_views.py 全2…
含む（部分一致）	`filter(employee_no__icontains=search_number)`		07 5. 実コード全解説：member_views.py 全2…

やりたいこと	書き方	ひとこと	出てくる所
並びのどれか	`filter(shop__in=['P', 'R'])`		07 5. 実コード全解説：member_views.py 全2…
つながった表の列でしぼる	`filter(name__shop='…')`		＋ 合わせて覚える
条件で値を作って並べる	`annotate(shop_order=Case(When(shop='異動・退社', then=Value(1)), default=Value(0), output_field=IntegerField())).order_by('shop_order')`		07 5. 実コード全解説：member_views.py 全2…
作って保存	`AsyncTask.objects.create(task_id=task_id, status='pending')`		06 16. 実コード全解説③：asynchronous_vie…
変えて保存	`obj.name = "…"` `obj.save()`		00 7. 実コード③：main_views.py の Login
まとめて変える	`member.objects.filter(…).update(shop='…')`	save() は呼ばれない	＋ 合わせて覚える
消す	`obj.delete()` `AsyncTask.objects.filter(…).delete()`		00 7. 実コード③：main_views.py の Login
あれば取り、無ければ作る	`obj, created = member.objects.get_or_create(employee_no=no, defaults={…})`		＋ 合わせて覚える
あれば直し、無ければ作る	`member.objects.update_or_create(employee_no=no, defaults={…})`	バックアップのロードに向く	＋ 合わせて覚える
必要な列だけ辞書で	`member.objects.values('employee_no', 'name')`		15 5. 不具合
1 列だけの並び	`values_list('employee_no', flat=True)`		＋ 合わせて覚える
つながった表を一緒に読む（速くする）	`Business_Time_graph.objects.select_related('name')`	何度も読みに行かない	＋ 合わせて覚える
全部成功か、全部取り消すか	`with transaction.atomic():`	ロードの途中で失敗した時に半端にしない	＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
重なりのエラー	`except IntegrityError:`		09 6. tasks.py 全解説①：工数バックアップ
保存の後に動かす（シグナル）	`@receiver(post_save, sender=member)`		06 13. デコレータ @
管理画面に出す	`admin.site.register(member)`		00 9. 実コード⑤：hozen_another/urls.py

Django（URL・ビュー・セッション・設定・ログ）

URL ごとに、答える係（ビュー）を決めます。設定は `hozen_another/settings.py` にあります。

やりたいこと	書き方	ひとこと	出てくる所
URL と係を結ぶ	`path('kosu_update/<int:pk>/', kosu_views.KosuUpdate.as_view(), name='kosu_update')`		00 6. 実コード②：kosu/urls.py
URL から数を受け取る	`<int:pk>`		00 6. 実コード②：kosu/urls.py
ほかの URL の決まりを取り込む	`path('api/', include('kosu.urls'))`		00 9. 実コード⑤：hozen_another/urls.py
React の画面を返す（どの URL でも）	`re_path(r'^(?!admin).*$', react_view)`	admin を除く。最後に書く	00 9. 実コード⑤：hozen_another/urls.py
URL から係の名前を調べる	`resolve(request.path).url_name`		07 11. 実コード全解説：asynchronous_view...
今の URL	`request.path`		07 11. 実コード全解説：asynchronous_view...
URL の ?○○= の値	`request.GET.get('task_id')`		07 11. 実コード全解説：asynchronous_view...
送られた本文（JSON）	`json.loads(request.body)`		00 7. 実コード③：main_views.py の Login

やりたいこと	書き方	ひとこと	出てくる所
ログインを覚える	`request.session['login_No'] = input_number`		00 7. 実コード③：main_views.py の Login
覚えた値を読む	`request.session.get('login_No')`		00 7. 実コード③：main_views.py の Login
ログアウトで忘れる	`request.session.flush()`	セッションを全部消す	＋ 合わせて覚える
JSON で返す	`return JsonResponse({'status': 'error', 'message': '…'}, status=400)`		00 7. 実コード③：main_views.py の Login
ファイルを返す（ダウンロード）	`FileResponse(open(path, 'rb'), as_attachment=True, filename=os.path.basename(path))`		07 11. 実コード全解説：asynchronous_view…
設定の値を読む	`from django.conf import settings` `settings.MEDIA_ROOT`		00 6. 実コード②：kosu/urls.py
.env の値を読む	`env = environ.Env()` `env.read_env(…)` `env('SECRET_KEY')`		00 12. 実コード⑦：settings.py の主要設定を1…
.env の真偽を読む	`DEBUG = env.bool('DEBUG')`		00 12. 実コード⑦：settings.py の主要設定を1…
本番で受け付けるドメイン	`ALLOWED_HOSTS = ['…']`	無いと 400	00 12. 実コード⑦：settings.py の主要設定を1…
POST を許すドメイン	`CSRF_TRUSTED_ORIGINS = ['https://…']`	無いと 403	01 11. settings.py を1行ずつ読む
ログの係を取る	`views_logger = logging.getLogger('views_logger')`		06 14. 実コード全解説①：main_views.py の …
ログに書く	`views_logger.info(message)` `.warning(…)` `.error(…)`		06 1. インデント ― Python最大の特徴

やりたいこと	書き方	ひとこと	出てくる所
エラーの詳しい場所もログに	`logger.exception("ロードに失敗しました")`	except の中で使う	15 4. 安全の問題
大きくなったら切り替えるログ	`RotatingFileHandler`		00 12. 実コード⑦：settings.py の主要設定を1...
リクエストの前後に必ず通す（ミドルウェア）	`def __call__(self, request): response = self.get_response(request)`		07 10. 実コード全解説：middleware と apps...
起動の時の準備	`class KosuConfig(AppConfig): def ready(self): import kosu.signals`		07 10. 実コード全解説：middleware と apps...
本番サーバーから呼ぶ入口	`application = get_wsgi_application()`		09 12. wsgi.py 全解説：Webサーバーとの玄関

Django REST framework（API）

画面（React）からの JSON の要求に答える API を作ります。

やりたいこと	書き方	ひとこと	出てくる所
API の係を作る	`class Login(APIView): def post(self, request):`		00 7. 実コード③：main_views.py の Login
関数で API を作る	`@api_view(['GET'])`		06 13. デコレータ @
受け取れる形を決める	`@parser_classes([MultiPartParser, JSONParser, FormParser])`	ファイルも受け取る	06 13. デコレータ @
返事を返す	`return Response(serializer.data, status=status.HTTP_200_OK)`		00 7. 実コード③：main_views.py の Login
返事の番号の名前	`status.HTTP_400_BAD_REQUEST`		00 7. 実コード③：main_views.py の Login
URL の ?○○= の値	`request.query_params.get('employee_no', None)`		06 7. 関数 def

やりたいこと	書き方	ひとこと	出てくる所
送られたデータ	`request.data`	JSON もフォームも	00 7. 実コード③：main_views.py の Login
送られたファイル	`request.FILES.get('file')`		07 11. 実コード全解説：asynchronous_view…
モデルと JSON を相互に変える	`class MemberSerializer(serializers.ModelSerializer): class Meta: model = member; fields = '__all__'`		07 6. 実コード全解説：serializers.py
全部の項目を使う	`fields = '__all__'`		07 6. 実コード全解説：serializers.py
入力を確かめる	`if serializer.is_valid(): serializer.save()`		07 5. 実コード全解説：member_views.py 全2…
確かめの誤り	`serializer.errors`		＋ 合わせて覚える
ページに分ける	`class CustomPagination(PageNumberPagination): page_size = 20`	管理者設定の menu_row があれば、その件数	06 9. クラスとインスタンス
そのページの分だけ取る	`result_page = paginator.paginate_queryset(members, request)`		07 5. 実コード全解説：member_views.py 全2…
ページつきで返す	`return paginator.get_paginated_response(serializer.data)`	count・next・results が入る	07 5. 実コード全解説：member_views.py 全2…

Excel を読む・書く（openpyxl・pandas）

バックアップ（DB → Excel）とロード（Excel → DB）で使います。

やりたいこと	書き方	ひとこと	出てくる所
新しい Excel を作る	`wb = openpyxl.Workbook()` `ws = wb.active`		09 6. tasks.py 全解説①：工数バックアップ
1 行足す	`ws.append(['従業員番号', '氏名'])`	見出しの行にも	09 6. tasks.py 全解説①：工数バックアップ
保存する	`wb.save(filepath)`		09 6. tasks.py 全解説①：工数バックアップ
Excel を読む	`wb = openpyxl.load_workbook(temp_file_path)`		09 8. tasks.py 全解説③：ロード
マスの値	`ws.cell(1, col).value`	1 から数える	09 8. tasks.py 全解説③：ロード
全部の行を値で読む	`for row in ws.iter_rows(min_row=2, values_only=True):`	2 行目から（見出しを飛ばす）	＋ 合わせて覚える
見出しが合っているか確かめる	`actual_headers = [ws.cell(1, col).value for col in range(1, len(expected_headers) + 1)]`		09 8. tasks.py 全解説③：ロード
表（データフレーム）を作る	`df = pd.DataFrame(data, columns=['工数定義区分', '作業詳細'])`		09 7. tasks.py 全解説②：予測データ出力
条件に合わない行を除く	`df = df[~df['工数定義区分'].isin(['', '#', '$'])]`	~ で「ではない」	09 7. tasks.py 全解説②：予測データ出力
重なった行を消す	`df.drop_duplicates(subset=['工数定義区分', '作業詳細'], keep='first')`		09 7. tasks.py 全解説②：予測データ出力
Excel に書く	`with pd.ExcelWriter(filepath, engine='openpyxl') as writer:` `df.to_excel(writer, index=False)`		09 7. tasks.py 全解説②：予測データ出力
Excel を表で読む	`df = pd.read_excel(path)`	簡単に読む時	＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
アップロードされたファイルを一時ファイルに書く	`for chunk in kosu_file.chunks(): temp_file.write(chunk)`		06 11. with 文

重い処理を裏で動かす（スレッド・タスク）

バックアップなどの重い処理は、別のスレッドで動かし、画面はタスクの状態を見に来ます。

やりたいこと	書き方	ひとこと	出てくる所
タスクの番号を作る	`task_id = str(uuid.uuid4())`		06 16. 実コード全解説③：asynchronous_vie…
タスクの状態を記録	`AsyncTask.objects.create(task_id=task_id, status='pending')`		06 16. 実コード全解説③：asynchronous_vie…
別のスレッドで動かす	`thread = threading.Thread(target=handle_task, args=(task_id, task_function, *args))` `thread.start()`		07 11. 実コード全解説：asynchronous_view…
スレッドごとの入れ物	`_thread_locals = local()`		06 18. 実コード全解説⑤：signals.py の差分計算
状態を見に行く（画面から）	`api.get(`/api/check_backup_status?task_id=${taskId}`)`	終わるまで少しおきに	07 7. 実コード全解説：urls.py
裏方の係を動かす（django-q）	`python manage.py qcluster`		01 9. バックエンド準備②：requirements.txt…

テスト（画面: Vitest と Testing Library）

`frontend/src/tests/` の `*.test.tsx`。`npm run test:run` で流します。

やりたいこと	書き方	ひとこと	出てくる所
テストのまとまり	`describe("Login Component", () => { … })`		12 4. 実コード全解説：Login.test.tsx 全13…
テストを 1 つ	`it("ログイン画面が表示される", () => { … })`		12 4. 実コード全解説：Login.test.tsx 全13…
毎回の準備	`beforeEach(() => { vi.clearAllMocks(); })`		12 4. 実コード全解説：Login.test.tsx 全13…
部品を描く	`render(<MemoryRouter><Login /></MemoryRouter>)`		12 4. 実コード全解説：Login.test.tsx 全13…
役割と名前で探す	`screen.getByRole("heading", { name: "ログイン" })`		12 4. 実コード全解説：Login.test.tsx 全13…
ラベルで入力欄を探す	`screen.getByLabelText("従業員番号")`		12 4. 実コード全解説：Login.test.tsx 全13…
出てくるまで待って探す	`await screen.findByRole("alert")`		12 4. 実コード全解説：Login.test.tsx 全13…
無いことを確かめる	`expect(screen.queryByRole("alert")).not.toBeInTheDocument()`	queryBy は無くても失敗しない	12 4. 実コード全解説：Login.test.tsx 全13…
文字を入れる	`fireEvent.change(input, { target: { value: "12345" } })`		12 4. 実コード全解説：Login.test.tsx 全13…

やりたいこと	書き方	ひとこと	出てくる所
押す	`fireEvent.click(button)`		12 4. 実コード全解説：Login.test.tsx 全13…
確かめる	`expect(x).toBe(5)`		12 4. 実コード全解説：Login.test.tsx 全13…
画面にある	`expect(el).toBeInTheDocument()`		12 4. 実コード全解説：Login.test.tsx 全13…
その文字がある	`expect(el).toHaveTextContent("…")`		12 4. 実コード全解説：Login.test.tsx 全13…
待ちながら確かめる	`await waitFor(() => { expect(…) })`		12 4. 実コード全解説：Login.test.tsx 全13…
部品をにせ物にする	`vi.mock("../../api/axios")`		12 4. 実コード全解説：Login.test.tsx 全13…
にせの関数	`vi.fn()`		12 4. 実コード全解説：Login.test.tsx 全13…
次の 1 回だけ成功させる	`mockedApi.post.mockResolvedValueOnce({ data: { status: "success" } })`		12 4. 実コード全解説：Login.test.tsx 全13…
次の 1 回だけ失敗させる	`mockedApi.post.mockRejectedValueOnce({ response: { status: 401 } })`		12 4. 実コード全解説：Login.test.tsx 全13…

やりたいこと	書き方	ひとこと	出てくる所
呼ばれたか・何で呼ばれたか	`expect(mockedNavigate).toHaveBeenCalledWith("/")`		12 4. 実コード全解説：Login.test.tsx 全13…
1 つのファイルだけ流す	`npx vitest run src/tests/MainPage/Login.test.tsx`		＋ 合わせて覚える

テスト（サーバー: Django）

`kosu/tests/` の `test_*.py`。`python manage.py test --settings=hozen_another.test_settings` で流します。

やりたいこと	書き方	ひとこと	出てくる所
テストのクラス	`class TestModels(BaseTestCase):`	TestCase を元にする	12 8. 実コード全解説：test_backend.py の土台
毎回の準備	`def setUp(self):`		12 8. 実コード全解説：test_backend.py の土台
テストを 1 つ	`def test_member_creation_and_str(self):`	名前は test_ で始める	12 9. 実コード全解説：TestModels
同じか	`self.assertEqual(response.status_code, 200)`		12 9. 実コード全解説：TestModels
真・偽か	`self.assertTrue(…)` `self.assertFalse(…)`		12 10. APIの 401 / 403 / 200 を確かめ…
入っているか	`self.assertIn('…', text)`		12 9. 実コード全解説：TestModels

やりたいこと	書き方	ひとこと	出てくる所
以下か	`self.assertLessEqual(AsyncTask.objects.count(), AsyncTask.MAX_RECORDS)`		12 9. 実コード全解説：TestModels
API を呼ぶ	`self.client = APIClient()` `self.client.post('/api/login/', {…}, format='json')`		12 8. 実コード全解説：test_backend.pyの土台
にせ物に取り替える	`with patch('kosu.views.….threading.Thread') as mock_thread:`		＋ 合わせて覚える
何を呼ばれても答えるにせ物	`MagicMock()`		12 8. 実コード全解説：test_backend.pyの土台
エラーが起きることを確かめる	`with self.assertRaises(ValueError):`		＋ 合わせて覚える

データベース（psql・SQL）

ふだんは ORM を使いますが、確かめの時に SQL を打つことがあります。

やりたいこと	書き方	ひとこと	出てくる所
データベースを作る	`psql -U postgres -c "CREATE DATABASE hozen_kosu;"`		01 12. バックエンド準備④：データベースを作ってmigr…
つなぐ	`psql -U postgres -d hozen_kosu`	`\q` で抜ける	01 12. バックエンド準備④：データベースを作ってmigr…
表の一覧	`\dt`	psql の中で	＋ 合わせて覚える
表の形	`\d kosu_member`	psql の中で	＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
行を見る	`SELECT * FROM kosu_member LIMIT 10;`		＋ 合わせて覚える
数える	`SELECT count(*) FROM kosu_business_time_graph;`		＋ 合わせて覚える
条件でしぼる	`SELECT … WHERE employee_no = 12345;`		＋ 合わせて覚える
データベースを丸ごと控える	`pg_dump -U postgres -d hozen_kosu -f backup.sql`	直す前に	＋ 合わせて覚える
控えから戻す	`psql -U postgres -d hozen_kosu -f backup.sql`		＋ 合わせて覚える

運用・デプロイ（本番に出す）

本番（Azure）へ出す前に、ローカルで動作とテストを確かめます。

やりたいこと	書き方	ひとこと	出てくる所
画面を本番用に作る	`npm run build`	frontend/ で	01 19. 本番ビルド
静的ファイルを集める	`python manage.py collectstatic`		09 3. 日常運用コマンド集
設計図を本番に当てる	`python manage.py migrate`	列を消す変更は特に注意	01 12. バックエンド準備④：データベースを作ってmigr…
本番のサーバーを動かす	`gunicorn hozen_another.wsgi:application`	Azure がこの形で起動	＋ 合わせて覚える
ログの最後を見る	`Get-Content web_console.log -Tail 50`		＋ 合わせて覚える
資料の PDF・Web 版を作り直す	`python _build_pdfs.py kosu kosu/README.md`	資料の置き場所で	09 17. 定期メンテナンスとドキュメントのPDF・Web版の…